

cornea__simple__mech__model

Axisymmetric hyperelastic finite-element model of the cornea under intraocular pressure (IOP), built on FEniCSx (dolfinx 0.11).

The `Simulation_Framework` package is a thin layer over dolfinx. It handles three things:

- material law and weak form,
- boundary conditions,
- outputs and plots.

With it, a simulation is written as a short script: a mesh, a JSON parameter file, a list of boundary conditions, and the outputs you want.

Author: B. Sauty

1. Installation (Docker)

FEniCSx is run inside a Docker container built from the provided `Dockerfile`. The image and the container are both named `fenicsx_cornea`.

Prerequisites

- Docker (Docker Desktop on Windows / macOS)

Build the image

From the root of the repository:

```
docker build -t fenicsx_cornea .
```

This starts from the official `dolfinx/dolfinx:v0.11.0` image, which provides dolfinx, PETSc, MPI and gmsh. It then adds:

- a LaTeX installation, so matplotlib can render text with LaTeX,
- OpenGL / Xvfb libraries for off-screen rendering with pyvista,
- the Python packages pandas, scipy, matplotlib, h5py, imageio, pyvista, jupyter and spyder-kernels.

Create and start the container

The repository is mounted as `/workspace` inside the container, so files are shared both ways: results written by the container appear in your local folder.

Linux / macOS:

```
docker run -it --name fenicsx_cornea -p 8888:8888 \
-v "$(pwd)":/workspace fenicsx_cornea
```

Windows (PowerShell):

```
docker run -it --name fenicsx_cornea -p 8888:8888 `
  -v "${PWD}:/workspace" fenicsx_cornea
```

To come back to the same container later:

```
docker start -ai fenicsx_cornea
```

Check the installation inside the container:

```
python3 -c "import dolfinx; print(dolfinx.__version__)"
```

Jupyter (optional)

Port 8888 is published, so a notebook server started inside the container is reachable from your browser:

```
jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
```

Then open the <http://127.0.0.1:8888/?token=...> link printed in the terminal.

Plotting without Docker

The plotting part (PlotManager) does not need dolfinx. To make plots on a machine without Docker, a plain Python environment is enough:

```
pip install numpy matplotlib h5py pillow
```

2. Repository structure

```
cornea_simple_mech_model/
|-- Dockerfile           # FEniCSx environment (image: fenicsx_cornea)
|-- README.md
|-- main_cornea_axi.py    # runs the IOP simulation (needs dolfinx)
|-- plot_cornea.py        # makes the figures from the results (no dolfinx)
|-- cornea.msh            # 2D (r, z) mesh of the corneal section, gmsh 2.2
|-- mech_params.json      # material parameters
|-- Simulation_Framework/
|   |-- __init__.py       # lazy exports (see below)
|   |-- mechanical_axisymmetric_framework.py # weak form, BCs, solver
|   |-- parameters_class.py # material parameters
|   |-- outputs_manager.py # XDMF fields + CSV scalars
|   |-- plots_manager.py  # reads results, plots them
`-- results/              # created by the run
    |-- cornea_IOP.xdmf / .h5 # fields, one time step per load step
    |-- cornea_IOP_scalars.csv # scalars, one row per load step
    `-- *.png, *.gif       # figures from plot_cornea.py
```

The Simulation_Framework package

Name	Module	Role
Hyperelastic_framework, mechanical_axisymmetric_framework		Displacement space (P2), axisymmetric kinematics, strain energy, residual, boundary conditions, Newton solver (PETSc SNES + MUMPS)
MechParams	parameters_class	Dataclass holding the material parameters. Checks that the parameters required by the chosen law are present
OutputManager, available_outputs	outputs_manager	Registry of field and scalar outputs, selected by name and written at each step
PlotManager, compare_scalars	plots_manager	Reads the XDMF/HDF5 and CSV files and makes scalar histories, field maps, line profiles and animations

Everything is importable from the package root:

```
from Simulation_Framework import (Hyperelastic_framework, OutputManager,
                                  available_outputs)
from Simulation_Framework import PlotManager, compare_scalars
```

Imports are **lazy**: a module is only loaded when one of its names is used. So `from Simulation_Framework import PlotManager` works on a machine without `dolfinx`. Inside the package, modules import each other with relative imports (`from .parameters_class import MechParams`).

3. Usage

All commands are run from the repository root. For the simulation, run them inside the container.

Run the simulation

```
python3 main_cornea_axi.py
```

In parallel:

```
mpirun -n 4 python3 main_cornea_axi.py
```

The script ramps the IOP from 0 to 2.0×10^{-3} MPa (15 mmHg) in 19 steps. It writes:

- the fields, `results/cornea_IOP.xdmf` (with its `.h5` data file),
- the scalars, `results/cornea_IOP_scalars.csv`,
- a quick plot of the apex displacement, `results/cornea_IOP_apex.png`.

Units are consistent: lengths in mm, stresses in MPa.

Make the figures

```
python3 plot_cornea.py
```

This works inside or outside the container. The XDMF file can also be opened in ParaView.

Material parameters: `mech_params.json`

```
{  
  "sedf_type": "Neo-Hookean",  
  "C_10": 0.05,  
  "C_01": 0.01,  
  "D": 0.001  
}
```

sedf_type	Strain energy density ψ	Required parameters
Neo-Hookean	$C_{10} (\bar{I}_1 - 3) + \frac{(J - 1)^2}{D}$	C_10, D
Mooney-Rivlin	$C_{10} (\bar{I}_1 - 3) + C_{01} (\bar{I}_2 - 3) + \frac{(J - 1)^2}{D}$	C_10, C_01, D
HGO	<i>declared in MechParams, not yet implemented in the framework</i>	C_10, D, k1, k2, kappa_disp

$\bar{I}_1 = \text{tr } \bar{\mathbf{C}}$ and $\bar{I}_2 = \frac{1}{2} [(\text{tr } \bar{\mathbf{C}})^2 - \text{tr}(\bar{\mathbf{C}}^2)]$ are the invariants of the isochoric right Cauchy-Green tensor $\bar{\mathbf{C}} = J^{-2/3} \mathbf{F}^T \mathbf{F}$. D controls the compressibility: the initial bulk modulus is $\kappa = 2/D$, and $D \rightarrow 0$ is the incompressible limit. Parameters not used by the chosen law are ignored, like C_01 for Neo-Hookean above.

Mesh

`cornea.msh` is a 2D mesh of the meridian section ($r \geq 0$, z the optical axis), in gmsh format 2.2. The framework relies on its physical tags:

Tag	Name	Default boundary condition in <code>main_cornea_axi.py</code>
1	anterior	free (zero traction)
2	posterior	follower pressure (IOP)
3	limbus	clamped in r and z
4	central_line	symmetry axis: $u_r = 0$
10	cornea	the domain

Writing a simulation

A minimal script follows the structure of `main_cornea_axi.py`:

```
from mpi4py import MPI
from dolfinx import fem
from dolfinx.io import gmshio
from petsc4py.PETSc import ScalarType
from Simulation_Framework import Hyperelastic_framework, OutputManager

mesh_data = gmshio.read_from_msh("cornea.msh", MPI.COMM_WORLD, 0, gdim=2)
mech = Hyperelastic_framework(mesh_data.mesh, "mech_params.json")

p = fem.Constant(mech.domain, ScalarType(0.0))
mech.build_BCs(mesh_data.facet_tags, [
    ["Dirichlet", 4, ("clamped", 0)],      # axis:  $u_r = 0$ 
    ["Dirichlet", 3, ("clamped", 0)],      # limbus:  $u_r = 0$ 
    ["Dirichlet", 3, ("clamped", 1)],      # limbus:  $u_z = 0$ 
    ["Pressure", 2, p],                    # IOP on the posterior face
])
mech.build_solver()

with OutputManager(mech, "results/my_run",
    fields=["displacement", "von_mises"],
    scalars=["apex_uz_anterior"]) as out:
    out.write(0.0)
    for n, value in enumerate([5e-4, 1e-3, 2e-3], start=1):
        p.value = value
        mech.solve_one_step(n, value)
        out.write(value)                # the "time" of the outputs is the load
```

The order matters. First create the framework, then call `build_BCs`, then `build_solver`, and only then create the `OutputManager`. The limbus reaction outputs need the facet tags set by `build_BCs`.

Boundary conditions `build_BC(facet_tags, boundary_conditions)`
takes a list of `[type, tag, values]`:

Type	values	Meaning
Dirichlet	("clamped", axis) or (Constant, axis)	Imposed displacement component, axis 0 = r , 1 = z
Pressure	Constant p	Follower pressure normal to the deformed surface, $p > 0$ pushes into the body
Slip	None or a constant vector $\mathbf{m}$	Sliding along the boundary ($\mathbf{u} \cdot \mathbf{N} = 0$) or $\mathbf{u} \cdot \mathbf{m} = 0$, by penalty
Neumann	traction vector (Constant)	Dead load
Neumann_follower	reference traction vector	Follower traction
Robin	(k , $\mathbf{u}_{\text{ref}}$)	Elastic support of stiffness k

Outputs

Outputs are chosen by name. `available_outputs()` prints the full list with descriptions.

Fields are written to `<basename>.xdmf`:

- displacement,
- stresses and strain: `cauchy_stress`, `PK1_stress`, `green_lagrange`,
- other quantities: `J`, `hoop_stretch`, `psi`, `von_mises`, `hydrostatic_pressure`.

Tensors are 3×3 in (r, z, θ) . All fields except `displacement` are cellwise constant (DG0).

Scalars are written to `<basename>_scalars.csv`, one row per step:

- apex displacements: `apex_uz_anterior`, `apex_uz_posterior`,
- `central_thickness`,
- reaction forces at the limbus: `limbus_reaction_r`, `limbus_reaction_z`,
- integrals over the whole cornea: `volume`, `strain_energy`.

Forces, volume and energy are for the full 3D cornea (integrated over 2π).

To add an output, add an entry to the `FIELDS` or `SCALARS` registry in `outputs_manager.py`. It then becomes available by name, and `PlotManager` picks it up automatically. For a nicer axis label, add a line to `LABELS` in `plots_manager.py`.

Plotting

```
import matplotlib.pyplot as plt
from Simulation_Framework import PlotManager, compare_scalars

pm = PlotManager("results/cornea_IOP")
pm.info() # what is in the files

# scalar histories: vs load, or one scalar vs another
pm.plot_scalars(["apex_uz_anterior", "apex_uz_posterior"])
pm.plot_scalars("limbus_reaction_z", x="apex_uz_anterior")

# maps on the (r, z) section
pm.plot_field("von_mises", deformed=True, mirror=True)
pm.plot_field("cauchy_stress", component="tt", symmetric=True) # hoop

# profiles along the axis r = 0: last step, then every load step
pm.plot_profile("green_lagrange", component="rr")
pm.plot_profile("green_lagrange", component="rr", t="all")

pm.animate_field("von_mises", "results/vm.gif", deformed=True, mirror=True)

# parameter study
runs = [PlotManager(f"results/run_{k}", label=f"run {k}") for k in range(3)]
compare_scalars(runs, "apex_uz_anterior")
plt.show()
```

Common arguments:

- `t` is the load step to show (by default the last one).
- `component` selects part of a vector or tensor:
 - vectors: 'r', 'z', 'magnitude',
 - tensors: 'rr', 'zz', 'tt', 'rz' and the other pairs.
- `deformed=True`, `scale=...` draws on the deformed configuration.
- `mirror=True` also draws the $r < 0$ half.
- `ax=...` draws into an existing matplotlib axis. Every method returns its axis.

For a quick look at every scalar history:

```
python3 -m Simulation_Framework.plots_manager results/cornea_IOP
```